

SESSION 9

EXPERIMENT 17 — UNIX FILE ABSTRACTION AND FILE DESCRIPTORS

Aim

To study the UNIX file abstraction and demonstrate the use of **file descriptors, file metadata, and basic file operations** using system calls.

Theory

In UNIX/Linux, many resources are treated as **files**. This is known as the **UNIX file abstraction**.

A file can represent regular files, directories, devices, pipes, sockets, etc.

File Descriptor

A **file descriptor (FD)** is a small integer used by a process to identify an open file.

The standard file descriptors are:

FD	Name	Purpose
0	stdin	Standard input
1	stdout	Standard output
2	stderr	Standard error

When a file is opened using `open()`, the operating system returns a file descriptor.

Important system calls

- `open()` — opens or creates a file.
- `read()` — reads data from a file.
- `write()` — writes data to a file.
- `close()` — closes a file descriptor.
- `stat()` — obtains file metadata.

File Metadata

Metadata contains information such as:

- File size
- File permissions
- Owner
- Group
- Number of links
- Last modification time
- File type

Algorithm

1. Create/open a file using `open()`.
2. Obtain the file descriptor.
3. Write data into the file using `write()`.
4. Close the file.
5. Use `stat()` to obtain metadata.
6. Display file size, permissions and inode number.
7. Reopen the file.
8. Read its contents using `read()`.
9. Display the contents.
10. Close the file.

C Program

```
C file_abstraction.c > main()
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <fcntl.h>
4  #include <unistd.h>
5  #include <sys/stat.h>
6
7  int main() {
8
9      int fd;
10     char buffer[200];
11     ssize_t bytes_read;
12
13     const char *filename = "sample.txt";
14     const char *data =
15         "Hello! This file demonstrates UNIX file abstraction.\n";
16
17     // Create and open file
18     fd = open(filename, O_CREAT | O_WRONLY | O_TRUNC, 0644);
19
20     if (fd == -1) {
21         perror("open");
22         return 1;
23     }
24
25     printf("File opened successfully.\n");
26     printf("File Descriptor: %d\n", fd);
27
28     // Write data
29     if (write(fd, data, 52) == -1) {
30         perror("write");
31         close(fd);
32         return 1;
33     }
34
35     printf("Data written successfully.\n");
36
37     close(fd);
38
39     // Obtain file metadata
40     struct stat file_info;
41
42     if (stat(filename, &file_info) == -1) {
```

```

42     if (stat(filename, &file_info) == -1) {
43         perror("stat");
44         return 1;
45     }
46
47     printf("\nFile Metadata:\n");
48     printf("Inode Number: %lu\n",
49         (unsigned long)file_info.st_ino);
50
51     printf("File Size: %ld bytes\n",
52         (long)file_info.st_size);
53
54     printf("Number of Links: %lu\n",
55         (unsigned long)file_info.st_nlink);
56
57     printf("File Permissions: %o\n",
58         file_info.st_mode & 0777);
59
60     // Reopen for reading
61     fd = open(filename, O_RDONLY);
62
63     if (fd == -1) {
64         perror("open");
65         return 1;
66     }
67
68     bytes_read = read(fd, buffer, sizeof(buffer) - 1);
69
70     if (bytes_read == -1) {
71         perror("read");
72         close(fd);
73         return 1;
74     }
75
76     buffer[bytes_read] = '\0';
77
78     printf("\nFile Contents:\n");
79     printf("%s", buffer);
80
81     close(fd);
82
83     return 0;
84 }

```

OUTPUT:

```
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ gcc file_abstraction.c -o file_abstraction
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ ./file_abstraction
File opened successfully.
File Descriptor: 3
Data written successfully.

File Metadata:
Inode Number: 34621422135783145
File Size: 52 bytes
Number of Links: 1
File Permissions: 777

File Contents:
Hello! This file demonstrates UNIX file abstraction.chakrika@Chakrika:/mnt/c/Users/panak/Downl
oads/OSSP PRACTICALS$
```

EXPERIMENT 18 — INODES, DIRECTORIES AND FILE LINKS

Aim

To study **inodes**, **directory entries**, **file attributes**, **hard links** and **symbolic links** and demonstrate their usage using Linux commands and system calls.

Theory

An **inode** is a data structure maintained by the Linux filesystem that stores information about a file.

An inode contains information such as:

- File type
- File permissions
- Owner
- File size
- Timestamps

- Number of hard links
- Pointers to data blocks

The **filename itself is not stored in the inode**. A directory contains entries that map filenames to inode numbers.

For example:

Directory

```
|  
+-- file1.txt → inode 1234  
|  
+-- file2.txt → inode 5678
```

Hard Link

A hard link is another directory entry pointing to the **same inode**.

```
original.txt ┌  
              └─> inode 1234  
hardlink.txt ┌
```

Therefore, both names refer to the same file data.

Command:

```
ln original.txt hardlink.txt
```

Symbolic Link

A symbolic link is a separate file that stores a **path to another file**.

```
symlink.txt → original.txt
```

Command:

```
ln -s original.txt symlink.txt
```

Unlike a hard link, a symbolic link has its **own inode**.

Important Linux Commands

ls -i

Displays inode numbers.

`ls -i`

stat

Displays detailed file metadata.

`stat original.txt`

ls -l

Displays permissions, owner, size, links and other information.

`ls -l`

Create hard link

`ln original.txt hardlink.txt`

Create symbolic link

`ln -s original.txt symlink.txt`

C Program — Inode and File Attributes

```
C inode_info.c > main()
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <sys/stat.h>
4
5  int main() {
6
7      struct stat file_info;
8      const char *filename = "sample.txt";
9
10     if (stat(filename, &file_info) == -1) {
11         perror("stat");
12         return 1;
13     }
14
15     printf("File Information\n");
16     printf("-----\n");
17
18     printf("File: %s\n", filename);
19
20     printf("Inode Number: %lu\n",
21         (unsigned long)file_info.st_ino);
22
23     printf("File Size: %ld bytes\n",
24         (long)file_info.st_size);
25
26     printf("Number of Hard Links: %lu\n",
27         (unsigned long)file_info.st_nlink);
28
29     printf("User ID: %u\n",
30         file_info.st_uid);
31
32     printf("Group ID: %u\n",
33         file_info.st_gid);
34
35     printf("Permissions: %o\n",
36         file_info.st_mode & 0777);
37
38     return 0;
39 }
```

OUTPUT:

```

chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ gcc inode_info.c -o inode_info

chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ ./inode_info
File Information
-----
File: sample.txt
Inode Number: 34621422135783145
File Size: 52 bytes
Number of Hard Links: 1
User ID: 1000
Group ID: 1000
Permissions: 777
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$

```

```

chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ ls -li sample.txt
34621422135783145 -rwxrwxrwx 1 chakrika chakrika 52 Oct  5 17:37 sample.txt
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ ln sample.txt hardlink.txt
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ ln -s sample.txt symlink.txt
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ ls -li sample.txt hardlink.txt
symlink.txt
34621422135783145 -rwxrwxrwx 2 chakrika chakrika 52 Oct  5 17:37 hardlink.txt
34621422135783145 -rwxrwxrwx 2 chakrika chakrika 52 Oct  5 17:37 sample.txt
24769797950706220 lrwxrwxrwx 1 chakrika chakrika 10 Oct  5 17:39 symlink.txt -> sample.txt
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ mkdir os_demo
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ cd os_demo
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS/os_demo$ echo "Operating System
s" > file.txt
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS/os_demo$ ls -li
total 0
14355223812686865 -rwxrwxrwx 1 chakrika chakrika 18 Oct  5 17:40 file.txt
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS/os_demo$ stat file.txt
  File: file.txt
  Size: 18          Blocks: 0          IO Block: 512   regular file
Device: 0,69   Inode: 14355223812686865   Links: 1
Access: (0777/-rwxrwxrwx)  Uid: ( 1000/chakrika)   Gid: ( 1000/chakrika)
Access: 2026-10-05 17:40:22.114168100 +0000
Modify: 2026-10-05 17:40:22.114168100 +0000
Change: 2026-10-05 17:40:22.114168100 +0000
 Birth: -
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS/os_demo$

```

Result

Thus, the Linux inode structure, file attributes, directory entries, hard links and symbolic links were successfully studied and demonstrated. It was observed that a hard link shares the same inode as the original file, whereas a symbolic link has a separate inode and stores a reference to the original file path.

